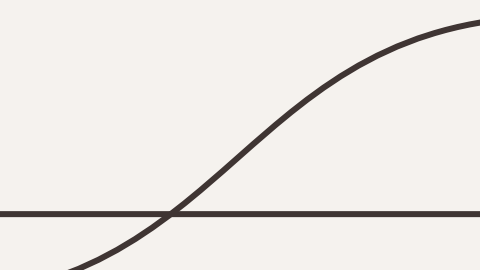


Data 2060 Hakimi Final Report

Team Members: Yesheng Guan, Yitian Liu, Yuchen Zhao



Multiclass Classification with Logistic Regression

This project tackles multiclass classification by adopting logistic regression as the model representation, which is decomposed into binary classification subtasks by two strategies:

One-vs-All (OvA) and One-vs-One (OvO).

Both strategies utilize cross-entropy loss to measure the divergence between predicted probabilities and true labels, while the Adam optimizer is employed to iteratively optimize model parameters.

Table of contents

01

Mathematical Background

Representation, Loss,
Optimizer

02

Numerical Technique

One-vs-All (OvA) and
One-vs-One (OvO)

03

Sklearn Results

Description

04

Summary

Description

01 Representation: The Logistic Unit

1. The Linear Combination (Raw Score)
 - a. Function: Aggregates input features using learned weights and bias
 - b. Limitation: The output z is unbounded, making it unsuitable for direct classification.

$$z = \mathbf{w}^T \mathbf{x} + b$$

Equation 1. The Linear Score Using Weighted sum of features

01 Representation: The Logistic Unit

2. The Activation Function (Sigmoid)

- a. Function: Applies the Sigmoid nonlinearity to the raw score z
- b. The output into the range $[0, 1]$, converting the score into an interpretable probability.
- c. A visualization of the function on the next page

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

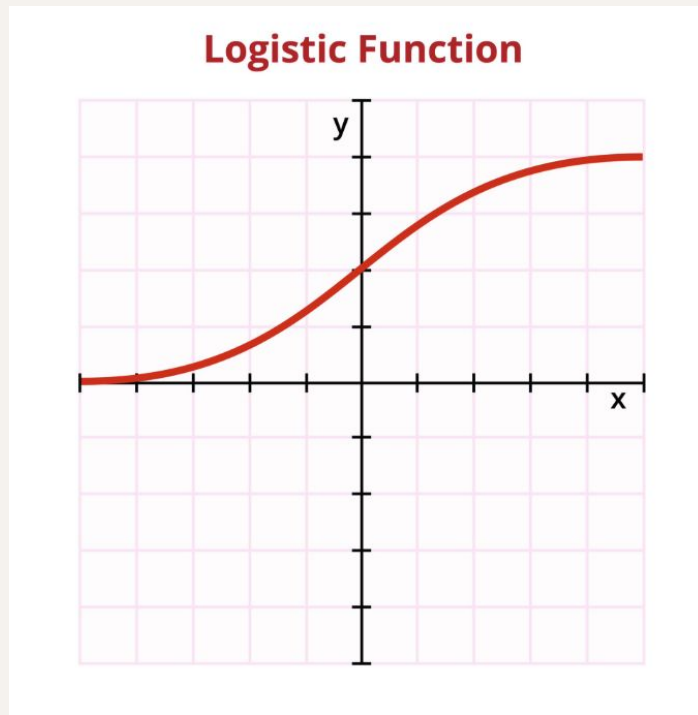
Equation 2. Sigmoid function maps score

01 Representation: The Logistic Unit

2. c. visualization

The sigmoid activation function transforms any real-valued input into a smooth output between 0 and 1.

The S-shaped curve compresses large positive or negative values, providing nonlinearity and enabling neural networks to learn complex decision boundaries.



01 Loss: Binary Cross-entropy (BCE)

$$\mathcal{L}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- a. BCE penalizes both false positives and false negatives.

The penalty increasing logarithmically as the predicted probability diverges from the true value.

Lower BCE loss indicates better model prediction.

- b. Limitations of BCE: not the most robust to outliers

When model predictions are extremely close to 0 or 1, the logarithm can approach negative infinity and causes undefined values and/or unstable training results.

01 Loss: L2 regularization

$$\ell(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|w\|_2^2$$

A regularizer is added to penalizes large weights. L2 is chosen to better fit the logistic regression model for its differentiability.

The intercept b is not regularized to prevent underfitting.

01 Optimizer: Newton_Raphson

NR is a second-order optimization algorithm for binary logistic regression. It uses gradient (first order) and Hessian matrix (second order) to update parameter.

Compared to SGD, it converges very fast and does not need to tune the learning rate. It uses adaptive step size (see next page)

The resourced demand for computing is larger than SGD, which means for large dataset may be not appropriate. However, our dataset is a small one.

01 Optimizer: Parameter update

$$\nabla L = \frac{1}{n} X_{aug}^T (p - y)$$

Where $p_i = \sigma(X_{aug,i} \cdot w)$

$$H = \frac{1}{n} X_{aug}^T \cdot S \cdot X_{aug}$$

$$S = \text{diag}(p \odot (1 - p))$$

$$H_{ij} = \frac{\partial^2 L(w)}{\partial w_i \partial w_j}$$

$$\Delta w = H^{-1} \cdot \nabla$$

Do not need learning rate!

The NR method uses Hessian matrix to update the parameter

02 From Binary to Multiclass: OvA & OvO

1. Strategy A: One-vs-All (OvA)
 - a. Logic: Trains K classifiers (where K is the number of classes).
 - b. Mechanism: Each classifier distinguishes "Class i " vs. "All Other Classes".
 - c. Prediction: Selects the class with the highest confidence score (probability).
 - d. Pros: Computational efficiency (fewer classifiers)

02 From Binary to Multiclass: OvA & OvO

Pseudo-code (OvA)

for each class cls:

Loop through each class

$y_{\text{binary}} = (y == \text{cls})$

 fit binary Newton logistic on (X, y_{binary})

Convert multiclass labels into binary labels

predict: compute p_{cls} for each class

Fit a binary logistic regression model

return argmax over classes

02 From Binary to Multiclass: OvA & OvO

2. Strategy B: One-vs-One (OvO)

- a. Logic: Trains $K(K-1)/2$ classifiers (all possible pairs).
- b. Mechanism: Each classifier distinguishes strictly between "Class A" vs. "Class B" (ignoring others)..
- c. Prediction: Uses a Voting Scheme—the class that wins the most determines the final prediction..
- d. Pros: Handles class imbalance better by focusing on local boundaries.

02 From Binary to Multiclass: OvA & OvO

Pseudo-code (OvO)

for each pair (c1, c2):

- restrict dataset to samples of c1 or c2
- relabel to binary, fit binary logistic

predict: each pair model votes for one class;
return class with most votes

Train binary classifiers for each pair

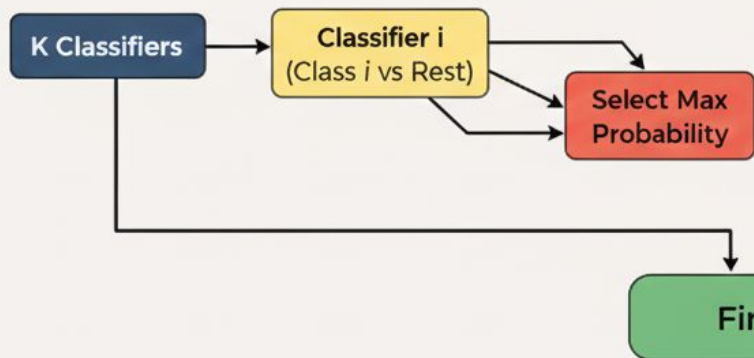
only use samples belonging to two classes, temporarily ignore all other classes in the dataset

Make predictions with all pairwise classifiers

Multiclass Classification Strategies

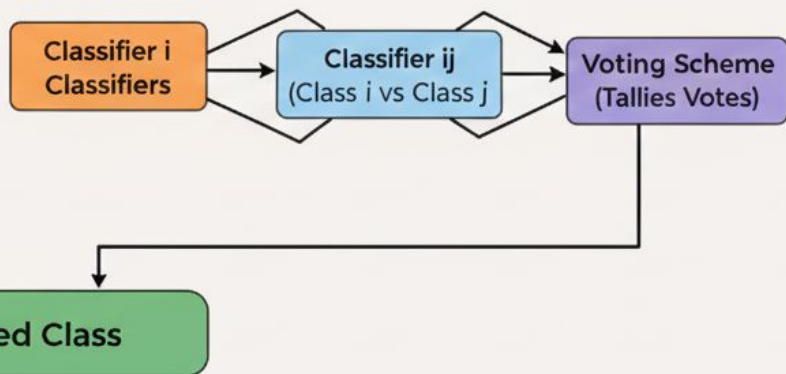
Strategy A: One-vs-All (OvA)

- a. Logic: K Classifiers
Logic: "Class i" vs. "All Others"
- b. Mechanism: "Class i" vs. "All Others" if Other Classes"
- Prediction: Highest Score
- d. Pros: Computational Efficiency (fewer classifiers)



Strategy B: One-vs-One (OvO)

- a. Trains $K(K-1)/2$ Classifiers
Mecanism: "Class i all possible pairs).
- b. Mechanism: Each classifies strictly between "Class A vs. "Class B" (ignoring others.. most aime the Final prediction..
- d. Pros: Handles class imbalance better by local boundaries



03 Verification & Sklearn Alignment

1. Setup
 - a. Objective: Validate the correctness of our "From-Scratch" Representation & Optimizer.
 - b. Benchmark: Compared against the official `sklearn.linear_model.LogisticRegression`.
 - c. Prediction Match: 100.00%, Our model predicts the exact same class labels as Sklearn for every single test sample.
 - d. Probability Difference: $8.88178e-16$ (Mean Absolute Error), The calculated probabilities are statistically aligned, with negligible variance due to floating point difference.

03 Verification & Sklearn Alignment

2. Dataset: the dataset used for testing is a dermatology dataset

Instances: 366 patients

Attributes: 34 ordinal features (12 clinical + 22 histopathological features, 1 binary feature (family history), 1 continuous feature (age)

Ordinal features are scored from 0–3, indicating severity

Missing values: Present only in the age attribute (8 cases, marked as “?”)

Goal: Predict one of six skin disease categories

03 Verification & Sklearn Alignment

3. Results

OvA:

Metric	Sklearn	Our Model	
Accuracy	0.9589	0.9589	
Precision	0.9594	0.9594	
Recall	0.9537	0.9537	
F1	0.9562	0.9562	

OvO:

Metric	Sklearn	Our Model	
Accuracy	0.9589	0.9589	
Precision	0.9515	0.9515	
Recall	0.9537	0.9537	
F1	0.9519	0.9519	

04 Summary

What was interesting:

- Exact Hessian use made the solver converge in very few iterations (quadratic convergence near optimum).
 - Numerical safeguards (clipping, tiny std, intercept exclusion from reg) are small but crucial to match high-precision results.
 - The explicit mapping $\lambda = 1/(C \cdot n)$ explains regularization scaling across implementations.
-

04 Summary

Challenges encountered:

- Ensuring the Hessian was well-conditioned; sometimes H was near-singular and required fallback steps.
 - Getting exact parity with sklearn required matching data splits, feature scaling, regularization handling, and solver tolerances.
 - Debugging divergence caused by missing intercept regularization or incorrect regularization scale.
 - Numerical underflow/overflow in $\exp$ required clipping thresholds (why we clipped at ± 709).
-

04 Summary

Practical recommendations:

- Always standardize features for second-order methods.
 - Seed pseudo-random generators for reproducible splits.
 - Check convergence by both parameter change and loss decrease.
 - Compare both metrics and parameter vectors to validate an implementation
-

Thanks
